1 Write a Python program to demonstrate Lists, Tuples, Sets, and Dictionaries with examples.

AIM

To write a Python program that shows the usage of Lists, Tuples, Sets, and Dictionaries with real-life examples.

ALGORITHM

1. Start the program.
2. Create a List to store multiple values and demonstrate operations.
3. Create a Tuple to store immutable data.
4. Create a Set to store unique values and demonstrate set operations.
5. Create a Dictionary to store key-value pairs.
6. Display results for each data structure.
7. End program.

CODE

```
# Demonstrating Lists, Tuples, Sets, and Dictionaries

# 1. LIST - Ordered, mutable collection
fruits = ["Apple", "Banana", "Mango", "Orange"]
print("List Example:", fruits)
fruits.append("Grapes")   # add element
print("After Append:", fruits)
print("Second Fruit:", fruits[1])  # indexing

# 2. TUPLE - Ordered, immutable collection
coordinates = (10, 20)
print("\nTuple Example:", coordinates)
print("X-coordinate:", coordinates[0])
print("Y-coordinate:", coordinates[1])
```

```
# 3. SET - Unordered, unique elements
unique_numbers = {1, 2, 3, 2, 4, 5}
print("\nSet Example:", unique_numbers)  # duplicates removed
unique_numbers.add(6)
print("After Adding 6:", unique_numbers)
print("Union with {5,6,7}:", unique_numbers.union({5, 6, 7}))

# 4. DICTIONARY - Key-Value pairs
student = {
    "name": "Yogin",
    "age": 20,
    "course": "Computer Science"
}
print("\nDictionary Example:", student)
print("Student Name:", student["name"])
student["age"] = 21  # update value
print("Updated Dictionary:", student)
```

OUTPUT

```
List Example: ['Apple', 'Banana', 'Mango', 'Orange']
After Append: ['Apple', 'Banana', 'Mango', 'Orange', 'Grapes']
Second Fruit: Banana

Tuple Example: (10, 20)
X-coordinate: 10
Y-coordinate: 20

Set Example: {1, 2, 3, 4, 5}
```

After Adding 6: {1, 2, 3, 4, 5, 6}

Union with {5,6,7}: {1, 2, 3, 4, 5, 6, 7}

Dictionary Example: {'name': 'Yogin', 'age': 20, 'course': 'Computer Science'}

Student Name: Yogin

Updated Dictionary: {'name': 'Yogin', 'age': 21, 'course': 'Computer Science'}

CONCLUSION

This program demonstrates:

- Lists → ordered, mutable collections (adding, indexing).
- Tuples → ordered, immutable collections (fixed coordinates).
- Sets → unordered collections with unique elements (union, add).
- Dictionaries → key-value pairs (update, access).

2 Write a program to perform list operations (append, insert, delete).

AIM

To write a Python program that performs basic list operations such as append, insert, and delete.

ALGORITHM

1. Start the program.
2. Create a list with some initial elements.
3. Use to add an element at the end of the list.
4. Use to add an element at a specific position.
5. Use or to delete an element.
6. Display the list after each operation.
7. End program.

CODE

```
# Program to perform list operations

# Initial list
numbers = [10, 20, 30, 40]
print("Initial List:", numbers)

# Append operation
numbers.append(50)
print("After Append (50):", numbers)

# Insert operation
numbers.insert(2, 25)  # insert 25 at index 2
print("After Insert (25 at index 2):", numbers)

# Delete operation
numbers.remove(40)  # remove element 40
print("After Delete (40):", numbers)

# Another delete using pop
numbers.pop(1)  # remove element at index 1
print("After Pop (index 1):", numbers)
```

OUTPUT

```
Initial List: [10, 20, 30, 40]
After Append (50): [10, 20, 30, 40, 50]
After Insert (25 at index 2): [10, 20, 25, 30, 40, 50]
After Delete (40): [10, 20, 25, 30, 50]
After Pop (index 1): [10, 25, 30, 50]
```

CONCLUSION

This program demonstrates:

- Append → adds an element at the end of the list.
- Insert → adds an element at a specific index.
- Delete/Remove/Pop → removes elements either by value or index.